

PONTIFICIA UNIVERSIDAD JAVERIANA

Facultad de Ingeniería

Introducción a la Inteligencia Artificial

Taller EDA – Fase 3

Modelo clásico de clasificación: Random Forest

Dataset: Adult (Census Income) – UCI Repository

Docente:

Ing. Julio Omar Palacio Niño, M.Sc.
palacio_julio@javeriana.edu.co

Presentado por:

Tomás Ramírez Roa
Juan Camilo Alba Castro
Johan Sebastian Mendez Ibarra
Andrés David Ortiz Forero

Objetivo:

Implementar, ajustar y evaluar un modelo clásico de Machine Learning basado en Random Forest para determinar si supera a la mejor arquitectura neuronal de la Fase 2 en la clasificación de ingresos del dataset Adult.

Bogotá D.C., 2026

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Contexto del problema de clasificación	4
4. Conexión con las fases anteriores	5
5. Justificación de la elección del modelo clásico	5
6. Fundamento teórico del Random Forest	6
6.1. Árboles de decisión	6
6.2. Impureza de Gini	6
6.3. Entropía	7
6.4. Ganancia de información	7
6.5. Sobreajuste en árboles individuales	7
6.6. Bagging	8
6.7. Selección aleatoria de variables	8
6.8. Votación mayoritaria	8
7. Hiperparámetros principales	8
8. Ventajas y limitaciones del Random Forest	9
8.1. Ventajas	9
8.2. Limitaciones	10
9. Metodología de implementación	10
10. Métricas de evaluación	11
10.1. Matriz de confusión	11
10.2. Accuracy	11
10.3. Precisión	11
10.4. Recall	12
10.5. F1-score	12

11. Entrenamiento y evaluación del Random Forest base	12
12. Ajuste de hiperparámetros	13
12.1. Espacio de búsqueda	13
12.2. Mejores hiperparámetros encontrados	14
13. Evaluación del Random Forest optimizado	15
14. Comparación con los modelos de la Fase 2	15
15. Comparación global de modelos	16
16. Discusión técnica	17
17. Conclusiones	17
18. Referencias	18

1. Introducción

Este informe corresponde a la entrega de la Fase 3 del Proyecto Final de Introducción a la Inteligencia Artificial. El objetivo de esta etapa es implementar, ajustar y evaluar un modelo clásico de Machine Learning para el problema de clasificación binaria sobre el dataset Adult del Repositorio UCI.

En la Fase 1 se realizó el análisis exploratorio, la limpieza, transformación y particionamiento del dataset. Posteriormente, en la Fase 2 se diseñaron, entrenaron y evaluaron tres arquitecturas de redes neuronales *feed-forward*. A partir de esa comparación, el Modelo B, correspondiente a una red neuronal con una capa oculta de 82 neuronas, fue seleccionado como la mejor arquitectura neuronal por presentar el mayor F1-score para la clase positiva >50K.

En esta tercera fase se selecciona Random Forest como modelo clásico de comparación. La finalidad es determinar si este algoritmo, basado en ensambles de árboles de decisión, logra superar el desempeño de la mejor red neuronal obtenida previamente.

La tarea predictiva consiste en determinar si una persona gana más de 50.000 dólares anuales o si gana una cantidad menor o igual a ese umbral, a partir de variables sociodemográficas, laborales y económicas del censo de Estados Unidos de 1994. Para mantener la continuidad metodológica, se usan exactamente los mismos conjuntos de entrenamiento y prueba generados en la Fase 1 y empleados en la Fase 2.

2. Objetivos

2.1. Objetivo general

Implementar, ajustar y evaluar un modelo clásico de Machine Learning basado en Random Forest para clasificar el nivel de ingreso del dataset Adult Census Income, comparando su rendimiento con la mejor red neuronal desarrollada en la Fase 2.

2.2. Objetivos específicos

- Explicar los fundamentos matemáticos y estadísticos que sustentan el algoritmo Random Forest.
- Entrenar un modelo Random Forest base utilizando los datos procesados y particionados en la Fase 1.

- Realizar un ajuste básico de hiperparámetros mediante validación cruzada con `GridSearchCV`.
- Evaluar el modelo utilizando matriz de confusión, Accuracy, Precisión, Recall y F1-score.
- Comparar el desempeño del Random Forest frente a las arquitecturas neuronales de la Fase 2.
- Determinar si el modelo clásico supera a la mejor red neuronal en términos de desempeño predictivo.

3. Contexto del problema de clasificación

El dataset Adult Census Income contiene información demográfica, laboral y socio-económica de personas registradas en datos censales de Estados Unidos de 1994. La variable objetivo es `income`, la cual indica si el ingreso anual de una persona es menor o igual a 50.000 USD, o si supera dicho umbral.

Para el proceso de modelado, la variable objetivo se codificó de la siguiente manera:

$$y = \begin{cases} 0, & \text{si el ingreso anual es } \leq 50K, \\ 1, & \text{si el ingreso anual es } > 50K. \end{cases} \quad (1)$$

Después del preprocesamiento realizado en la Fase 1, el dataset quedó transformado en 82 variables predictoras numéricas o binarias. Este resultado proviene de la eliminación de la columna redundante `education`, el tratamiento de valores nulos, la codificación *one-hot* de variables categóricas y la normalización de variables numéricas.

La tabla 1 resume las dimensiones de los conjuntos utilizados en esta fase.

Tabla 1: Dimensiones de los conjuntos utilizados en la Fase 3

Conjunto	Observaciones	Variables predictoras
X_{train}	39.073	82
X_{test}	9.769	82
y_{train}	39.073	1
y_{test}	9.769	1

La distribución de clases presenta un desbalance moderado: aproximadamente el 76,1 % de los registros pertenece a la clase $\leq 50K$, mientras que el 23,9 % pertenece a la clase $> 50K$. Este desbalanceo condiciona la elección de métricas de evaluación, ya que la exactitud bruta puede ser engañosa si el modelo favorece excesivamente la clase mayoritaria.

4. Conexión con las fases anteriores

La Fase 3 no parte desde cero. Su objetivo es cerrar el proceso iniciado en las dos fases anteriores. En la Fase 1 se construyó una base limpia, codificada, normalizada y particionada. En la Fase 2 se entrenaron tres redes neuronales sobre esa misma base. En esta fase se evalúa un modelo clásico bajo las mismas condiciones experimentales.

En la Fase 2 se implementaron tres arquitecturas:

- **Modelo A:** Perceptrón.
- **Modelo B:** red neuronal con una capa oculta de 82 neuronas.
- **Modelo C:** red neuronal con dos capas ocultas de dos neuronas cada una.

El Modelo B fue seleccionado como la mejor red neuronal, debido a que obtuvo el mayor F1-score para la clase positiva $> 50K$. Ese resultado se usa como punto de referencia para determinar si Random Forest logra superar el desempeño de las redes neuronales.

5. Justificación de la elección del modelo clásico

Para la Fase 3 se eligió Random Forest como modelo clásico de Machine Learning. Esta elección se justifica por las siguientes razones técnicas:

- El dataset Adult es un conjunto tabular con variables numéricas y variables binarias generadas mediante codificación *one-hot*.
- Random Forest suele tener buen desempeño en problemas de clasificación tabular.
- Permite capturar relaciones no lineales entre las variables predictoras y la variable objetivo.
- Reduce el riesgo de sobreajuste frente a un único árbol de decisión.

- Permite trabajar con muchas variables predictoras.
- Incorpora el parámetro `class_weight="balanced"` para compensar el desbalance de clases.

En comparación con otros modelos clásicos, Random Forest ofrece un equilibrio favorable. KNN puede verse afectado por la alta dimensionalidad generada por la dumbificación; SVM puede tener un costo computacional elevado con este tamaño de datos; Naive Bayes depende de la suposición fuerte de independencia condicional; y un único árbol de decisión puede presentar alta varianza y sobreajuste. Random Forest representa una opción robusta, flexible y técnicamente adecuada para comparar contra las redes neuronales de la Fase 2.

6. Fundamento teórico del Random Forest

6.1. Árboles de decisión

Random Forest se basa en la construcción de múltiples árboles de decisión. Un árbol de decisión es un modelo supervisado que divide el espacio de características mediante reglas sucesivas. Cada nodo interno representa una condición sobre una variable, cada rama representa una decisión y cada hoja representa una clase final.

En clasificación binaria, el árbol busca separar las observaciones de acuerdo con la variable objetivo. Para ello, en cada nodo selecciona la variable y el punto de corte que producen la mayor pureza en los nodos resultantes.

6.2. Impureza de Gini

Uno de los criterios más utilizados para dividir nodos es la impureza de Gini. Para un nodo t con K clases, se define como:

$$Gini(t) = 1 - \sum_{k=1}^K p_k^2 \quad (2)$$

donde p_k es la proporción de observaciones de la clase k dentro del nodo.

Para el caso binario del dataset Adult, con clases $\leq 50K$ y $> 50K$, la fórmula se expresa como:

$$Gini(t) = 1 - p_0^2 - p_1^2 \quad (3)$$

Un nodo completamente puro tiene impureza cero. Esto ocurre cuando todas las observaciones pertenecen a una sola clase.

6.3. Entropía

Otro criterio de partición es la entropía, que mide el grado de incertidumbre dentro de un nodo:

$$Entropia(t) = - \sum_{k=1}^K p_k \log_2(p_k) \quad (4)$$

Para el caso binario:

$$Entropia(t) = -p_0 \log_2(p_0) - p_1 \log_2(p_1) \quad (5)$$

Cuando un nodo contiene una mezcla equilibrada de clases, la entropía es alta. Cuando el nodo contiene principalmente una sola clase, la entropía disminuye.

6.4. Ganancia de información

La ganancia de información mide cuánto se reduce la impureza después de dividir un nodo:

$$Ganancia = I(t) - \left(\frac{n_L}{n} I(t_L) + \frac{n_R}{n} I(t_R) \right) \quad (6)$$

donde $I(t)$ es la impureza del nodo original, $I(t_L)$ e $I(t_R)$ son las impurezas de los nodos hijo izquierdo y derecho, y n , n_L , n_R son sus respectivos tamaños.

El árbol selecciona la división que maximiza la reducción de impureza.

6.5. Sobreajuste en árboles individuales

Un árbol de decisión individual puede crecer demasiado y memorizar patrones específicos del conjunto de entrenamiento. Esto produce sobreajuste: buen rendimiento en entrenamiento, pero menor capacidad de generalización sobre datos nuevos.

Este problema se agrava cuando el árbol tiene profundidad ilimitada, las hojas contienen pocas observaciones, existen muchas variables predictoras o hay ruido en los datos. Random Forest surge como una alternativa para reducir este riesgo.

6.6. Bagging

Random Forest utiliza una técnica llamada *bagging*, abreviatura de *Bootstrap Aggregating*. Esta consiste en entrenar múltiples árboles sobre muestras distintas del conjunto de entrenamiento, obtenidas mediante muestreo con reemplazo.

El objetivo del bagging es reducir la varianza del modelo. Un único árbol puede ser inestable y sobreajustarse; en cambio, un conjunto de árboles entrenados sobre muestras diferentes tiende a producir una predicción más estable.

6.7. Selección aleatoria de variables

Además del muestreo aleatorio de observaciones, Random Forest introduce aleatoriedad en la selección de variables. En cada división de un árbol no se consideran todas las variables predictoras, sino un subconjunto aleatorio. Esto evita que todos los árboles sean demasiado similares y mejora la capacidad de generalización del bosque.

6.8. Votación mayoritaria

En clasificación, cada árbol emite un voto por una clase. La clase con más votos se convierte en la predicción final:

$$\hat{y} = \text{moda}(h_1(x), h_2(x), \dots, h_B(x)) \quad (7)$$

donde $h_b(x)$ representa la predicción del árbol b y B corresponde al número total de árboles.

7. Hiperparámetros principales

El comportamiento de Random Forest depende de varios hiperparámetros. La tabla 2 resume los más relevantes para este proyecto.

Tabla 2: Hiperparámetros principales de Random Forest

Hiperparámetro	Descripción e impacto
<code>n_estimators</code>	Número de árboles del bosque. Un mayor número de árboles suele mejorar la estabilidad del modelo, aunque aumenta el costo computacional.
<code>max_depth</code>	Profundidad máxima de cada árbol. Una profundidad alta permite capturar relaciones complejas, pero puede aumentar el riesgo de sobreajuste.
<code>min_samples_split</code>	Número mínimo de observaciones requeridas para dividir un nodo interno. Valores más altos hacen que los árboles sean más conservadores.
<code>min_samples_leaf</code>	Número mínimo de observaciones que debe contener una hoja. Valores mayores reducen la posibilidad de memorizar casos específicos.
<code>criterion</code>	Criterio usado para medir la calidad de una división. En este proyecto se consideran Gini y Entropía.
<code>class_weight</code>	Permite asignar pesos a las clases. Se utiliza <code>balanced</code> para compensar el desbalance entre $\leq 50K$ y $> 50K$.
<code>random_state</code>	Semilla aleatoria que permite reproducir los resultados.
<code>n_jobs</code>	Número de núcleos de procesamiento utilizados. Con <code>-1</code> , se emplean todos los núcleos disponibles.

8. Ventajas y limitaciones del Random Forest

8.1. Ventajas

- Tiene buen desempeño en problemas tabulares con variables heterogéneas.
- Captura relaciones no lineales entre variables.
- Reduce el sobreajuste en comparación con un único árbol de decisión.
- No requiere supuestos fuertes sobre la distribución de las variables.
- Permite trabajar con muchas variables predictoras.

- Es robusto frente a ruido moderado y valores atípicos.
- Permite incorporar pesos de clase para tratar desbalance.

8.2. Limitaciones

- Es menos interpretable que un único árbol de decisión.
- Puede requerir más recursos computacionales si se usan muchos árboles.
- No extrapola bien fuera del rango observado en los datos de entrenamiento.
- Su desempeño depende de la selección adecuada de hiperparámetros.
- Aunque reduce el sobreajuste, no lo elimina completamente.

9. Metodología de implementación

La implementación se realizó en Python utilizando la librería Scikit-learn. No se repitió el preprocesamiento de la Fase 1. Se utilizaron directamente los archivos generados en dicha fase:

- `X_train.csv`
- `X_test.csv`
- `y_train.csv`
- `y_test.csv`

Esto garantiza que Random Forest se entrene y evalúe sobre exactamente el mismo particionamiento utilizado para las redes neuronales de la Fase 2. Esta condición es necesaria para que la comparación entre modelos sea justa.

El procedimiento seguido fue:

1. Cargar los conjuntos de entrenamiento y prueba generados en la Fase 1.
2. Entrenar un Random Forest base con `class_weight="balanced"`.
3. Evaluar el Random Forest base sobre el conjunto de prueba.
4. Realizar un ajuste de hiperparámetros mediante `GridSearchCV`.

5. Entrenar el Random Forest optimizado con la mejor combinación encontrada.
6. Evaluar el modelo optimizado sobre el conjunto de prueba.
7. Comparar Random Forest base, Random Forest optimizado y los tres modelos neuronales de la Fase 2.

10. Métricas de evaluación

Se emplearon las mismas métricas utilizadas en la Fase 2, lo que permite una comparación directa entre modelos.

10.1. Matriz de confusión

La matriz de confusión organiza los resultados en verdaderos negativos, falsos positivos, falsos negativos y verdaderos positivos.

Tabla 3: Estructura de la matriz de confusión

	Predicción $\leq 50K$	Predicción $> 50K$
Real $\leq 50K$	TN	FP
Real $> 50K$	FN	TP

10.2. Accuracy

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (8)$$

Mide la proporción total de predicciones correctas. Sin embargo, no es la métrica principal en este proyecto debido al desbalance de clases.

10.3. Precisión

$$Precision = \frac{TP}{TP + FP} \quad (9)$$

Indica qué proporción de las predicciones positivas fueron correctas.

10.4. Recall

$$Recall = \frac{TP}{TP + FN} \quad (10)$$

Indica qué proporción de los casos reales positivos logró detectar el modelo.

10.5. F1-score

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (11)$$

El F1-score combina Precisión y Recall mediante una media armónica. Es la métrica principal de comparación porque penaliza tanto los falsos positivos como los falsos negativos, siendo especialmente adecuada en presencia de desbalance de clases.

11. Entrenamiento y evaluación del Random Forest base

Se entrenó un primer Random Forest con parámetros base e incluyendo `class_weight="balance"` para compensar el desbalance de clases.

La tabla 4 presenta las métricas obtenidas.

Tabla 4: Métricas del Random Forest base

Métrica	Valor
Accuracy	0,8563
Precisión >50K	0,7420
Recall >50K	0,6125
F1 >50K	0,6710
F1 macro	0,7895
F1 weighted	0,8513

La matriz de confusión del modelo base se presenta en la tabla 5.

Tabla 5: Matriz de confusión – Random Forest base

	Predicción $\leq 50K$	Predicción $> 50K$
Real $\leq 50K$	6.933	498
Real $> 50K$	906	1.432

El Random Forest base obtiene el Accuracy más alto entre los modelos evaluados hasta este punto, con un valor de 0,8563. También presenta una Precisión de 0,7420 para la clase $>50K$, lo que indica que, cuando predice ingresos superiores a 50.000 USD, suele acertar con frecuencia.

Sin embargo, su Recall de 0,6125 indica que deja sin detectar una parte importante de los casos reales positivos. De los 2.338 casos reales de la clase $>50K$, el modelo identifica correctamente 1.432 y clasifica incorrectamente 906 como $\leq 50K$. Por tanto, aunque el modelo base es preciso, todavía no logra el mejor equilibrio entre Precisión y Recall para la clase positiva.

12. Ajuste de hiperparámetros

Con el objetivo de mejorar el desempeño del modelo base, se realizó un ajuste de hiperparámetros mediante `GridSearchCV`. Esta técnica evalúa distintas combinaciones de parámetros usando validación cruzada y selecciona aquella con mejor desempeño promedio.

La métrica utilizada durante la búsqueda fue `f1`. Esta decisión se justifica porque el dataset presenta desbalance de clases: la clase $>50K$ representa solo el 23,9 % de los registros. En este contexto, optimizar únicamente Accuracy podría favorecer modelos que clasifiquen correctamente la clase mayoritaria $\leq 50K$, pero que fallen al detectar la clase positiva. El F1-score permite equilibrar Precisión y Recall para la clase $>50K$.

12.1. Espacio de búsqueda

La tabla 6 presenta los hiperparámetros evaluados.

Tabla 6: Espacio de búsqueda del GridSearchCV

Hiperparámetro	Valores evaluados
<code>n_estimators</code>	100, 200
<code>max_depth</code>	None, 10, 20
<code>min_samples_split</code>	2, 5
<code>min_samples_leaf</code>	1, 2
<code>criterion</code>	gini, entropy

El total de combinaciones evaluadas fue:

$$2 \times 3 \times 2 \times 2 \times 2 = 48 \quad (12)$$

Como se utilizó validación cruzada con $cv = 3$, el procedimiento entrenó:

$$48 \times 3 = 144 \quad (13)$$

modelos durante la búsqueda.

12.2. Mejores hiperparámetros encontrados

La mejor combinación encontrada se presenta en la tabla 7.

Tabla 7: Mejores hiperparámetros encontrados para el Random Forest optimizado

Hiperparámetro	Valor seleccionado
<code>criterion</code>	gini
<code>max_depth</code>	None
<code>min_samples_leaf</code>	2
<code>min_samples_split</code>	2
<code>n_estimators</code>	200

El mejor F1 promedio en validación cruzada fue de 0,7060. La selección de `max_depth=None` indica que el modelo se benefició de árboles sin restricción explícita de profundidad, mientras que `min_samples_leaf=2` ayudó a evitar hojas demasiado específicas. Además, el uso de 200 árboles incrementó la estabilidad del ensamble.

13. Evaluación del Random Forest optimizado

Una vez seleccionados los mejores hiperparámetros, se evaluó el Random Forest optimizado sobre el conjunto de prueba. Los resultados se presentan en la tabla 8.

Tabla 8: Métricas del Random Forest optimizado

Métrica	Valor
Accuracy	0,8383
Precisión >50K	0,6263
Recall >50K	0,8037
F1 >50K	0,7040
F1 macro	0,7964
F1 weighted	0,8445

La matriz de confusión del Random Forest optimizado se muestra en la tabla 9.

Tabla 9: Matriz de confusión – Random Forest optimizado

	Predicción $\leq 50K$	Predicción $> 50K$
Real $\leq 50K$	6.326	1.105
Real $> 50K$	459	1.879

El ajuste de hiperparámetros produjo una mejora significativa en el F1-score de la clase >50K: pasó de 0,6710 en el modelo base a 0,7040 en el modelo optimizado. Esta mejora se explica principalmente por el aumento del Recall, que pasó de 0,6125 a 0,8037.

En términos prácticos, el modelo optimizado detectó correctamente 1.879 de los 2.338 casos reales de la clase >50K, mientras que el modelo base detectaba 1.432. Esto representa 447 casos positivos adicionales correctamente clasificados.

El costo de esta mejora fue una disminución en la Precisión, que pasó de 0,7420 a 0,6263, y un aumento en los falsos positivos. Sin embargo, bajo el criterio principal de F1-score para la clase positiva, el modelo optimizado es superior.

14. Comparación con los modelos de la Fase 2

La tabla 10 resume las métricas oficiales de los modelos neuronales de la Fase 2.

Tabla 10: Métricas de los modelos neuronales de la Fase 2

Modelo	Accuracy	Precisión >50K	Recall >50K	F1 >50K	F1 macro	F1 weighted
A – Perceptrón	0,8311	0,7082	0,5004	0,5865	0,7402	0,8203
B – 1 capa oculta (82 n.)	0,8076	0,5654	0,8473	0,6782	0,7705	0,8186
C – 2 capas ocultas (2 n.)	0,7999	0,5534	0,8494	0,6702	0,7633	0,8118

El Modelo B fue seleccionado como la mejor red neuronal porque obtuvo el mayor F1-score para la clase positiva entre las arquitecturas neuronales, con un valor de 0,6782. Este valor se utiliza como referencia principal para determinar si Random Forest logra superar el desempeño de las redes neuronales.

15. Comparación global de modelos

La tabla 11 presenta la comparación global entre los modelos neuronales, el Random Forest base y el Random Forest optimizado. La tabla está ordenada de mayor a menor F1-score para la clase >50K.

Tabla 11: Comparación global de modelos

Modelo	Accuracy	Precisión >50K	Recall >50K	F1 >50K	F1 macro	F1 weighted
Random Forest optimizado	0,8383	0,6263	0,8037	0,7040	0,7964	0,8445
B – 1 capa oculta (82 n.)	0,8076	0,5654	0,8473	0,6782	0,7705	0,8186
Random Forest base	0,8563	0,7420	0,6125	0,6710	0,7895	0,8513
C – 2 capas ocultas (2 n.)	0,7999	0,5534	0,8494	0,6702	0,7633	0,8118
A – Perceptrón	0,8311	0,7082	0,5004	0,5865	0,7402	0,8203

El Random Forest optimizado obtiene el mayor F1-score para la clase >50K, con un valor de 0,7040. Esto supera al Modelo B de la Fase 2, que alcanzó 0,6782. Por tanto, bajo el criterio principal del proyecto, el Random Forest optimizado es el mejor modelo global.

Es importante observar que el Random Forest base obtuvo el mayor Accuracy y la mayor Precisión para la clase positiva. Sin embargo, su bajo Recall limitó su F1-score. El ajuste de hiperparámetros redujo la Precisión, pero aumentó considerablemente el Recall, generando un mejor equilibrio entre ambas métricas.

16. Discusión técnica

Los resultados muestran que la elección del mejor modelo depende de la métrica utilizada. Si se considera únicamente el Accuracy, el Random Forest base parece ser el mejor modelo, con 0,8563. Sin embargo, debido al desbalance de clases del dataset, esta métrica no es suficiente para decidir el modelo final.

El Perceptrón obtuvo un Accuracy competitivo, pero su Recall para la clase >50K fue de apenas 0,5004. Esto significa que dejó sin detectar aproximadamente la mitad de los casos reales de ingresos superiores a 50.000 USD. Esta limitación es coherente con su naturaleza lineal.

El Modelo B fue la mejor red neuronal, con un F1-score de 0,6782 para la clase positiva. Su principal fortaleza fue el alto Recall, con 0,8473. Esto indica que detectó una gran proporción de los casos positivos, aunque con una Precisión menor que la del Random Forest.

El Modelo C alcanzó el Recall más alto entre las redes neuronales, con 0,8494. Sin embargo, su Precisión fue menor, lo que produjo un mayor número de falsos positivos. Esto se explica por su arquitectura altamente restringida: dos capas ocultas con solo dos neuronas cada una, lo que genera un cuello de botella fuerte frente a un espacio de entrada de 82 dimensiones.

El Random Forest base presentó un comportamiento más conservador. Su Precisión fue alta, pero su Recall fue bajo. En otras palabras, cuando predijo >50K acertó con frecuencia, pero dejó sin detectar muchos casos positivos reales.

El Random Forest optimizado logró el mejor equilibrio general. Su F1-score para la clase >50K fue de 0,7040, superando a todos los modelos evaluados. Aunque su Recall fue menor que el de las redes neuronales B y C, su Precisión fue mayor, lo que produjo el mejor balance entre ambas métricas.

Desde una perspectiva técnica, este resultado muestra que, para un dataset tabular como Adult, un modelo clásico de ensamble puede superar a arquitecturas neuronales simples cuando se realiza un ajuste adecuado de hiperparámetros.

17. Conclusiones

1. Random Forest fue una elección adecuada para la Fase 3, debido a su buen desempeño en datos tabulares, su capacidad para capturar relaciones no lineales y su menor tendencia al sobreajuste frente a un único árbol de decisión.

2. El Random Forest base obtuvo el mayor Accuracy de todos los modelos evaluados, con 0,8563, y también la mayor Precisión para la clase >50K, con 0,7420. Sin embargo, su Recall de 0,6125 limitó su F1-score para la clase positiva.
3. El ajuste de hiperparámetros mediante `GridSearchCV` mejoró significativamente el desempeño del modelo bajo el criterio principal del proyecto. El F1-score para la clase >50K pasó de 0,6710 en el modelo base a 0,7040 en el modelo optimizado.
4. La mejora del Random Forest optimizado se explica principalmente por el aumento del Recall para la clase positiva, que pasó de 0,6125 a 0,8037. Esto permitió detectar 447 casos positivos adicionales en el conjunto de prueba.
5. El mejor modelo neuronal de la Fase 2 fue el Modelo B, con un F1-score de 0,6782 para la clase >50K. Sin embargo, el Random Forest optimizado superó este valor con un F1-score de 0,7040.
6. Bajo el criterio principal de F1-score para la clase positiva, el Random Forest optimizado es el mejor modelo global del proyecto.
7. La comparación final demuestra que un modelo clásico de ensamble puede superar a redes neuronales *feed-forward* simples en un problema tabular, siempre que se realice un ajuste adecuado de hiperparámetros y se evalúe con métricas coherentes con el desbalance de clases.

18. Referencias

1. BREIMAN, Leo. Random forests. *Machine Learning*, vol. 45, n.º 1, p. 5–32, 2001.
2. HASTIE, Trevor; TIBSHIRANI, Robert y FRIEDMAN, Jerome. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. New York: Springer, 2009.
3. GÉRON, Aurélien. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3a ed. Sebastopol: O'Reilly Media, 2022. ISBN 978-1-098-12597-4.
4. PEDREGOSA, Fabian, et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, vol. 12, p. 2825–2830, 2011.

5. BACHE, K.; LICHMAN, M. UCI Machine Learning Repository: Adult Data Set. Irvine: University of California, School of Information and Computer Science, 1996. Disponible en: <https://archive.ics.uci.edu/ml/datasets/adult>
6. INSTITUTO COLOMBIANO DE NORMAS TÉCNICAS Y CERTIFICACIÓN (ICONTEC). *NTC 1486: Documentación, presentación de tesis, trabajos de grado y otros trabajos de investigación*. 6a ed. Bogotá: ICONTEC, 2008.